

Tutorial 04: Attribute Values (Tweening) - Teaching Guide

Overview

This tutorial moves away from the legacy `glVertexPointer`/`glColorPointer` API and replaces it with **custom vertex attributes** - the modern, shader-friendly way to supply per-vertex data. We also introduce a **tween animation** that smoothly morphs geometry between two shapes every frame using a uniform fraction.

Prerequisites and Setup

Installation

Make sure you have completed the previous tutorials and have the virtual environment activated:

```
# Activate virtual environment (if not already active)
.\shader_tutorial_venv\Scripts\Activate.ps1

# Navigate to tutorials folder
cd tutorials

# Run tutorial 04
python "04_AttributeValues(Tweening).py"
```

Expected Output: A window with two colourful triangles that continuously and smoothly morph (stretch/squish) between two different shapes in a looping ping-pong animation.

Interactive Controls:

- **ESC** - Exit
-

Learning Objectives

By the end of this tutorial, students will understand:

1. What vertex attributes are and how they differ from uniforms
 2. How to define multiple attributes in a GLSL vertex shader
 3. How to lay out interleaved data in a VBO
 4. `glGetAttribLocation`, `glEnableVertexAttribArray`, `glVertexAttribPointer`
 5. The GLSL `mix()` function for position interpolation (tweening)
 6. How to drive a smooth animation with a per-frame uniform value
 7. Why attribute-based rendering replaces legacy pointer functions
-

What's New in This Tutorial

Compared to Tutorial 03, we're adding:

- ☒ **Custom vertex attributes** (not just legacy `gl_Vertex`/`gl_Color`)
- ☒ **Two position attributes per vertex** (start + end shape)
- ☒ `glVertexAttribPointer` replaces `glVertexPointer`/`glColorPointer`
- ☒ `glGetAttribLocation` to resolve attribute slots
- ☒ **Tween animation** - smooth geometry morphing driven by a uniform
- ☒ **Interleaved VBO layout** with 9 floats per vertex record

Legacy vs Modern Attribute Approach

What the legacy API looked like (Tutorials 01–03):

```
glEnableClientState(GL_VERTEX_ARRAY)
glEnableClientState(GL_COLOR_ARRAY)
glVertexPointer(3, GL_FLOAT, 24, vbo)      # implicit "gl_Vertex"
glColorPointer(3, GL_FLOAT, 24, vbo + 12)  # implicit "gl_Color"
```

These functions only work with the **fixed-function pipeline's predefined slots** (position, colour, normal, tex-coord). You cannot create your own.

What we do now (Tutorial 04):

```
glEnableVertexAttribArray(position_loc)
glEnableVertexAttribArray(tweened_loc)
glEnableVertexAttribArray(color_loc)
glVertexAttribPointer(position_loc, 3, GL_FLOAT, False, stride, vbo)
glVertexAttribPointer(tweened_loc, 3, GL_FLOAT, False, stride, vbo + 12)
glVertexAttribPointer(color_loc, 3, GL_FLOAT, False, stride, vbo + 24)
```

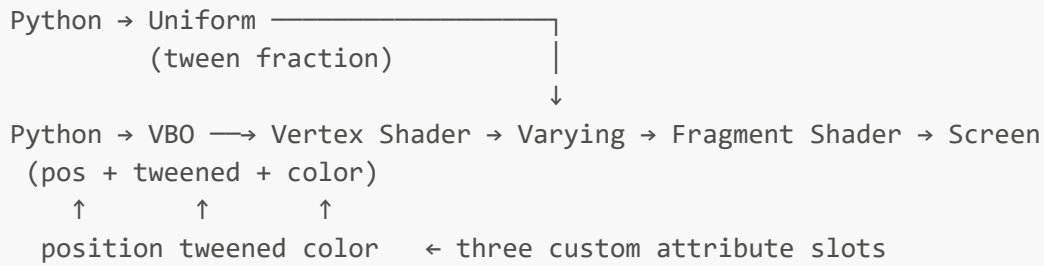
We use **arbitrary attribute slots identified by name** - the shader declares them, Python queries their location, then tells the GPU which bytes of the VBO feed which slot.

Data Flow Comparison

Tutorial 03 (Fog with Uniforms):

```
Python → Uniforms —————
      (fog settings)
                        |
                        ↓
Python → VBO → Vertex Shader → Varying → Fragment Shader → Screen
      (pos + color)
```

Tutorial 04 (Tweening with Attributes):



Code Breakdown

1. Vertex Shader — Attribute Declarations

```
uniform float tween;

attribute vec3 position;
attribute vec3 tweened;
attribute vec3 color;

varying vec4 baseColor;

void main() {
    gl_Position = gl_ModelViewProjectionMatrix * mix(
        vec4( position, 1.0 ),
        vec4( tweened, 1.0 ),
        tween
    );
    baseColor = vec4(color, 1.0);
}
```

Key points:

Keyword	What it means
<code>attribute</code>	Per-vertex data fed from a VBO. Each vertex gets its own value.
<code>uniform</code>	Same value for every vertex in the draw call (set once from Python).
<code>varying</code>	Output from vertex shader; interpolated and forwarded to fragment shader.

2. Understanding `attribute`

```
attribute vec3 position;
```

- **Declared in** the vertex shader only (fragment shaders cannot have attributes)
- **One value per vertex** - when the GPU processes vertex 0 it reads **position** from row 0 of the VBO, vertex 1 reads row 1, etc.
- **Read-only** inside the shader
- **Not interpolated** between vertices (unlike **varying**)

Attribute vs Uniform vs Varying:

	Attribute	Uniform	Varying
Scope	One per vertex	Same for all vertices	Vertex → Fragment
Declared in	Vertex shader	Any shader	Both shaders
Source	VBO	CPU code	Written by vertex shader
Interpolated?	No	No	Yes
Examples	Position, colour, normal, UV	Time, fog dist, light pos	Colour, UV, fog factor

3. The GLSL **mix()** Function for Tweening

```
gl_Position = gl_ModelViewProjectionMatrix * mix(
    vec4( position, 1.0 ),
    vec4( tweened, 1.0 ),
    tween
);
```

mix(a, b, t) computes:

$$\text{result} = a * (1 - t) + b * t$$

With our tween fraction:

tween	mix(position, tweened, tween)	Meaning
0.0	position	Original shape
0.5	halfway between the two	Mid-morph
1.0	tweened	Alternate shape

Visual (top vertex):

```
position = (0, 1, 0)   tweened = (1, 3, 0)

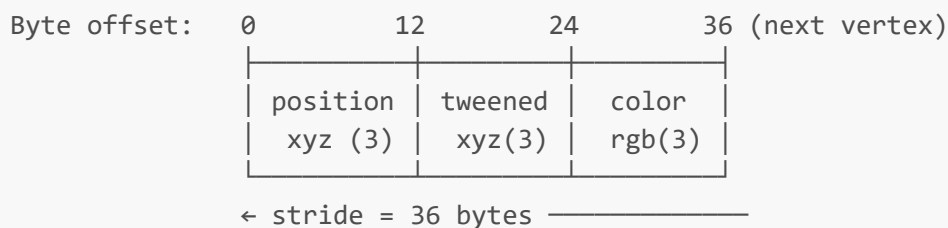
tween=0.0: vertex at (0, 1, 0)   ← original
tween=0.5: vertex at (0.5, 2, 0) ← halfway up
tween=1.0: vertex at (1, 3, 0)   ← fully "tweened"
```

This interpolation happens **on the GPU per vertex** - extremely fast even for large meshes!

4. VBO Layout — Interleaved Attributes

```
self.vbo = vbo.VBO(
    np.array([
        # position      tweened      color
        [ 0, 1, 0, 1, 3, 0, 0, 1, 0 ],
        [-1, -1, 0, -1, -1, 0, 1, 1, 0 ],
        ...
    ], dtype='f')
)
```

Each row is one vertex. In memory, the 9 floats are laid out back-to-back:



`stride = 9 * 4 = 36` bytes tells the GPU to jump 36 bytes between consecutive vertices.

5. Querying Attribute Locations

```
self.position_location = glGetAttribLocation(self.shader, 'position')
self.tweened_location  = glGetAttribLocation(self.shader, 'tweened')
self.color_location    = glGetAttribLocation(self.shader, 'color')
self.tween_location    = glGetUniformLocation(self.shader, 'tween')
```

Why query?

- The compiler assigns attribute slots in an unspecified order
- `glGetAttribLocation` returns the integer slot index for a named attribute
- `glGetUniformLocation` returns the integer slot index for a named uniform

- Both return `-1` if the name is not found (typo check!)

Always query after `compileProgram()` - the locations do not exist before compilation.

Difference: `Attrib` vs `Uniform` lookup:

Function	For	Example return
<code>glGetAttribLocation(shader, name)</code>	<code>attribute</code> variables	<code>0, 1, 2 ...</code>
<code>glGetUniformLocation(shader, name)</code>	<code>uniform</code> variables	<code>0, 1, 2 ...</code>

6. Enabling Attribute Arrays

```
glEnableVertexAttribArray(self.position_location)
glEnableVertexAttribArray(self.tweened_location)
glEnableVertexAttribArray(self.color_location)
```

- Tells the GPU: "read this attribute from an array (VBO), not a constant"
- Without this, the attribute gets a default constant value (usually `0`)
- Must be called **while the VBO is bound**
- Must be **disabled after drawing** (else later draw calls may read stale/invalid memory)

7. `glVertexAttribPointer` — Describing the Data Layout

```
stride = 9 * 4    # 9 floats x 4 bytes = 36 bytes

glVertexAttribPointer(
    self.position_location,    # which attribute slot
    3,                        # how many components (x, y, z)
    GL_FLOAT,                 # data type
    False,                    # normalize? No
    stride,                    # bytes per vertex record
    self.vbo                   # byte offset into VBO (offset 0)
)
glVertexAttribPointer(
    self.tweened_location,
    3, GL_FLOAT, False, stride, self.vbo + 12    # offset 12 bytes
)
glVertexAttribPointer(
    self.color_location,
    3, GL_FLOAT, False, stride, self.vbo + 24    # offset 24 bytes
)
```

Parameter guide:

Parameter	Meaning
location	Integer slot returned by <code>glGetAttribLocation</code>
size	Number of components (1–4). Here 3 = <code>vec3</code>
type	<code>GL_FLOAT</code> , <code>GL_INT</code> , <code>GL_UNSIGNED_BYTE</code> , etc.
normalized	If True, integers are mapped to [0,1] or [-1,1]
stride	Bytes from start of one vertex to start of next (0 = tightly packed)
pointer	Byte offset of this attribute's first element in the VBO

Diagram — how offsets map into the VBO:

Vertex 0:		Vertex 1:
[pos.x][pos.y][pos.z][tw.x][tw.y][tw.z][r][g][b] [pos.x]...		
↑ 0 bytes	↑ 12 bytes	↑ 24 bytes
position_loc	tweened_loc	color_loc

8. Passing the Tween Uniform Each Frame

```
glUniform1f(self.tween_location, self.tween_fraction)
```

- This is the same `glUniform1f` used in Tutorial 03
- Called inside `Render()` so the value updates every frame
- `tween_fraction` changes over time to drive the animation

9. The Tween Animation Loop

```
def update_tween(self, dt):
    self._anim_time = (self._anim_time + dt) % self.anim_duration
    frac = self._anim_time / self.anim_duration    # 0.0 → 1.0

    # Ping-pong: fold back so it goes 0→1→0
    if frac > 0.5:
        frac = 1.0 - frac
    frac *= 2.0

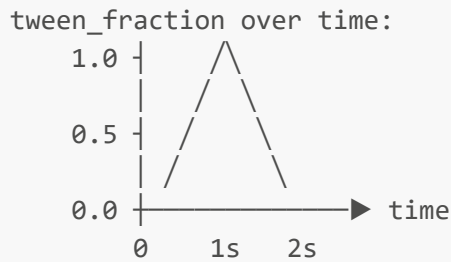
    self.tween_fraction = frac
```

The ping-pong fold-back:

```
raw frac over time:    0 → 0.5 → 1.0 → 0 → ...

fold-back step 1:      0 → 0.5 → 0.5 → 0 → ... (mirror after 0.5)
multiply by 2:         0 → 1.0 → 0.0 → 0 → ... (rescale to full range)
```

Result: The geometry smoothly stretches out to its "tweened" shape, then smoothly returns to the original, looping continuously.



Complete Data Flow — One Vertex Through the Pipeline

Python (per frame):

```
# Vertex 0 in VBO: position=(0,1,0)  tweened=(1,3,0)  color=(0,1,0)
# Uniform:         tween = 0.6
```

Vertex Shader (vertex 0):

```
// position = (0, 1, 0)
// tweened  = (1, 3, 0)
// color    = (0, 1, 0)  ← green
// tween    = 0.6

mix(vec4(0,1,0,1), vec4(1,3,0,1), 0.6)
  = vec4(0,1,0,1) * 0.4 + vec4(1,3,0,1) * 0.6
  = vec4(0.6, 2.2, 0.0, 1.0)  ← interpolated position

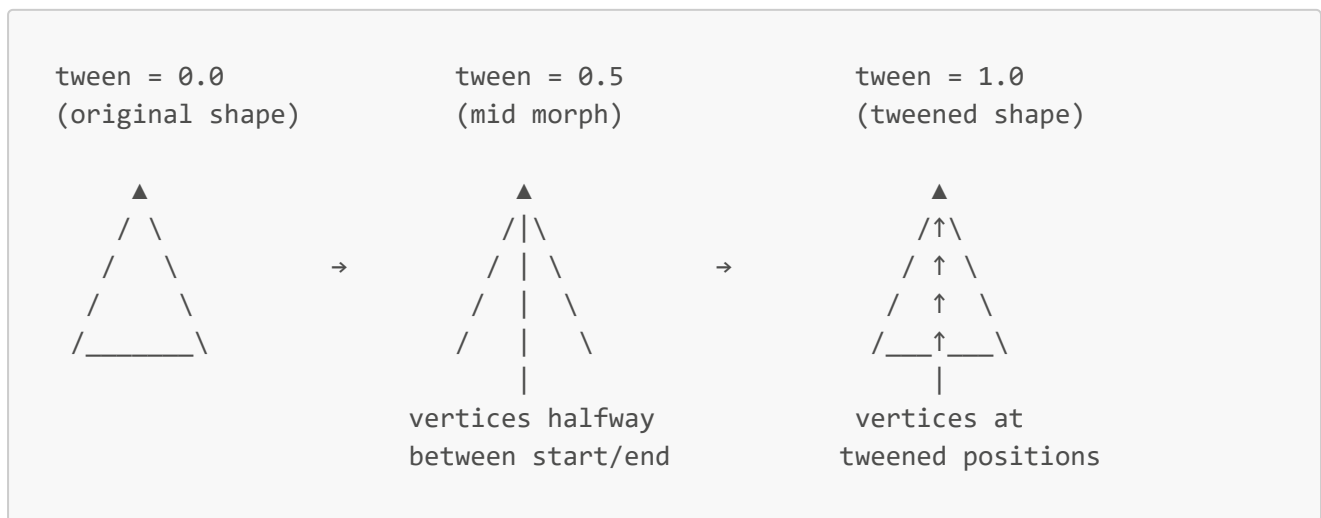
gl_Position = ModelViewProjectionMatrix * vec4(0.6, 2.2, 0.0, 1.0)
baseColor   = vec4(0, 1, 0, 1)  ← still green
```

Rasterisation: Creates fragments, interpolates `baseColor` across the triangle.

Fragment Shader:

```
gl_FragColor = baseColor;  // (0, 1, 0, 1) → green pixel
```


Visual Explanation of Tweening



Real-world use cases:

- **Character animation:** morph between idle pose and walk pose key-frames
- **Facial animation:** morph from neutral expression to smile
- **Fluid simulation:** smooth transition between fluid states
- **UI animations:** shape morphing in interface elements

Comparison: Legacy Pointers vs Attribute Pointers

Aspect	Legacy (glVertexPointer)	Modern (glVertexAttribPointer)
Attribute count	Fixed (position, colour, normal, UV)	Unlimited (any name/type)
Shader integration	Implicit (gl_Vertex , gl_Color)	Explicit (named, queried)
Custom data	Not possible	Yes — store anything per vertex
Multiple positions	Not possible	Yes — two or more positions per vertex
API	glEnableClientState + glVertexPointer	glEnableVertexAttribArray + glVertexAttribPointer
Modern OpenGL	Deprecated	Recommended

Built-in GLSL Functions Used

mix(a, b, t)

```
genType mix(genType a, genType b, float t)
```

- Returns: $a * (1-t) + b * t$
- Works on `float`, `vec2`, `vec3`, `vec4`
- $t=0 \rightarrow a$, $t=1 \rightarrow b$, $t=0.5 \rightarrow$ midpoint

Examples:

```
mix(0.0, 10.0, 0.3) // → 3.0
mix(vec3(1,0,0), vec3(0,0,1), 0.5) // → vec3(0.5, 0, 0.5) purple
mix(vec4(pos,1), vec4(tweened,1), tween) // position interpolation
```

Common Student Questions

Q: Why do we need `glEnableVertexAttribArray`?

A: The GPU needs to know whether to read from an array (VBO) or use a constant value for each attribute. `glEnableVertexAttribArray` activates array mode for that slot. Without it, all vertices receive the default value `(0, 0, 0, 1)`.

Q: What happens if I forget to `glDisableVertexAttribArray`?

A: The attribute array pointer remains active. If a later draw call uses a VBO that is too small, the GPU may read memory past the end of it, causing a crash (segfault) or garbage rendering.

Q: Can I store more than two positions per vertex?

A: Yes! Add more `attribute vec3` declarations (`position1`, `position2`, `position3`...) and corresponding VBO columns. You can then blend between them with any weights you like using `mix()` multiple times or a custom formula.

Q: Why not just update the VBO every frame instead of tweening in the shader?

A: Uploading a new VBO every frame (CPU-side) is expensive — it requires transferring data over the CPU-GPU bus. Tweening on the GPU only uploads one float (the `tween` uniform) per frame, which is negligible. This is the core advantage of GPU-side animation.

Q: Can attributes be types other than `vec3`?

A: Yes! Attributes can be `float`, `vec2`, `vec3`, `vec4`, and even matrices (`mat2`–`mat4`). The `size` parameter in `glVertexAttribPointer` (1–4) matches the number of components.

Q: What if `glGetAttribLocation` returns `-1`?

A: The attribute name was not found in the compiled shader. Common causes: typo in the name string, the attribute was optimised away by the compiler (unused attributes are removed), or the program was not linked yet.

Experiments to Try

1. Change the Tween Duration

```
self.anim_duration = 0.5 # Fast morph
self.anim_duration = 5.0 # Slow, dramatic morph
```

2. One-Way Tween (No Ping-Pong)

```
def update_tween(self, dt):
    self._anim_time = (self._anim_time + dt) % self.anim_duration
    self.tween_fraction = self._anim_time / self.anim_duration
```

Effect: Geometry snaps back to original at the end of each cycle.

3. Non-Linear Tween (Ease In/Out)

In the vertex shader, replace `tween` with a smoothed version:

```
float smoothTween = tween * tween * (3.0 - 2.0 * tween); // smoothstep
gl_Position = gl_ModelViewProjectionMatrix * mix(
    vec4(position, 1.0),
    vec4(tweened, 1.0),
    smoothTween
);
```

Effect: Slow start, fast middle, slow end — much more natural animation.

4. Colour Tween

Store two colours per vertex and blend those too:

```
attribute vec3 color;
attribute vec3 color2;
...
baseColor = vec4(mix(color, color2, tween), 1.0);
```

Effect: Colours morph alongside the geometry.

5. Three-Way Morph

```
attribute vec3 position;
attribute vec3 tweened1;
attribute vec3 tweened2;
```

```
uniform float tween;

void main() {
    vec3 phase1 = mix(position, tweened1, clamp(tween * 2.0, 0.0, 1.0));
    vec3 phase2 = mix(phase1, tweened2, clamp(tween * 2.0 - 1.0, 0.0, 1.0));
    gl_Position = gl_ModelViewProjectionMatrix * vec4(phase2, 1.0);
}
```

Effect: Three-shape animation sequence.

Debugging Tips

Problem: Geometry doesn't move / animation frozen

- Check `tween_fraction` is actually changing (add a print statement)
- Verify `self.tween_location != -1` (location found)
- Confirm `glUniform1f` is called after `glUseProgram`

Problem: Black/invisible geometry

- Check `color_location != -1`
- Verify `glEnableVertexAttribArray` is called for all attributes
- Print attribute locations to confirm they're not all `-1`

Problem: Crash / segfault on draw

- Check `stride` is correct ($9 \times 4 = 36$ bytes)
- Verify byte offsets in `glVertexAttribPointer` are correct
- Ensure `glDisableVertexAttribArray` runs even on exception (use `finally`)

Problem: Weird stretching / wrong morph

- Check VBO data layout — tweened column must be columns 3-5 (bytes 12-23)
- Print a few rows of the VBO array to verify structure

Visualise the tween value:

```
// Temporarily replace baseColor to see tween gradient
baseColor = vec4(tween, 0.0, 1.0 - tween, 1.0); // blue→red as tween increases
```

Performance Considerations

Why GPU tweening is efficient:

Approach	Data per frame	CPU work	GPU work
----------	----------------	----------	----------

Approach	Data per frame	CPU work	GPU work
Update VBO each frame	$n_vertices \times 3 \times 4$ bytes	Build new array, upload	Normal vertex processing
GPU tween (this tutorial)	4 bytes (one float)	None	<code>mix()</code> per vertex

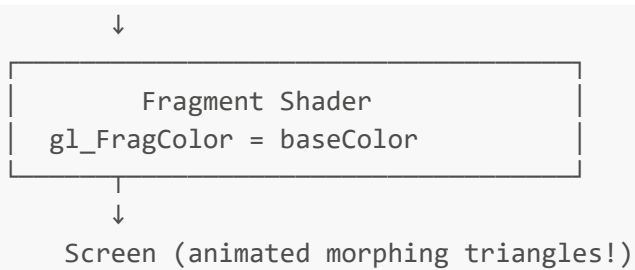
For a character with 10,000 vertices, GPU tweening uploads 4 bytes instead of 120,000 bytes each frame.

Extending to key-frame animation:

A real game character stores many key-frame poses in the VBO. The animation system picks the two surrounding key-frames and sets two attribute pointers to those frames' data columns. The GPU interpolates between them in real time — this is exactly how early game engines handled character animation.

Summary Diagram





Next Steps

You now know:

- ☒ Tutorial 01: Basic shaders and VBOs
- ☒ Tutorial 02: Varying values and colour interpolation
- ☒ Tutorial 03: Uniform values and fog calculations
- ☒ Tutorial 04: Custom attributes and tween animation

Concepts unlocked for future tutorials:

- Multiple VBO buffers (one per attribute)
- Texture coordinates as attributes
- Normal vectors for lighting
- Vertex Array Objects (VAOs) — bundle all attribute setup into one object
- Instanced rendering — draw many copies with per-instance attributes
- Geometry shaders — generate geometry on the GPU

Congratulations!

You have now moved **entirely away from the legacy OpenGL API**. The combination of:

1. **Attributes** — arbitrary per-vertex data (position, colour, UV, normals...)
2. **Uniforms** — settings passed from CPU each frame (time, transform, light pos...)
3. **Varyings** — interpolated values flowing from vertex to fragment shader

...gives you complete control over the GPU pipeline. Everything modern OpenGL (and Vulkan/Metal/WebGPU) builds on these three foundations.